Delicut

Designing a meal subscription that works even when life does not go to plan

Delicut is a UAE-based meal-plan service used by 15,000+ customers. I worked on its web subscription journey and mobile app, helping turn a complex set of meal, delivery and subscription rules into an experience that felt simple to use.

TL;DR

Delicut already had a strong service. The product problem was what happened when customers stopped following the “perfect” journey.

People joined plans late. They skipped meals. They paused several days. They changed delivery addresses. They came back after their plan had expired. Some days were editable, while others were locked because a cut-off had passed.

Many of these situations did not yet have a clear product experience.

I helped map these states, simplify the rules behind them and design web and mobile flows that responded clearly to where the customer was in their subscription.

The result was a more flexible product system built around real customer behaviour rather than one ideal path.

Project at a glance

Product
Meal-plan subscription and wellness platform

Platforms
Web + iOS + Android

Customers
15,000+ across the UAE

My role
Product / UX Designer

Focus
Subscription UX, onboarding, information architecture, delivery management, mobile experience and design QA

Timeline
[Add exact project dates]

Team
Design team, client stakeholders, product and engineering

The solution at a glance

I worked across two connected parts of the product.

On the web

I helped simplify the journey from:

Landing page → plan selection → personalisation → delivery details → review → payment

Instead of asking people to create an account early, we moved toward a softer e-commerce-style sign-up. Contact information could be collected naturally as part of delivery and checkout.

This kept people moving toward their meal plan instead of stopping them with account setup before they had made a decision.

On mobile

The challenge was different.

Once someone had bought a plan, the app needed to understand what was happening **today**, what had already happened, what could still be changed and what was already locked.

I helped design the app around these changing subscription states.

A customer might open the same app while:

their plan has not started yet

their plan is active

today's delivery is already locked

tomorrow's meals can still be changed

yesterday's meal can be rated

several deliveries have been skipped

their plan is about to expire

their plan has already expired

Instead of designing each screen as an isolated page, we began treating Delicut as a system of states.

Overview

Delicut delivers ready-made, macro-tracked meals across the UAE.

Customers choose a meal plan based on their goals and preferences, receive meals on scheduled days and manage those deliveries through the app.

The company wanted to make this experience easier for people who were new to structured meal plans.

That meant the product had two jobs.

It had to help someone confidently **choose and buy the right plan**.

Then it had to help them **live with that plan every day**.

The second part became much more complex than it first appeared.

The problem

The product worked best when the customer did exactly what we expected.

The original experience followed a clean path:

Choose a plan.
Start the plan.
Receive meals.
Finish the plan.

Real customers behaved differently.

They skipped one day.

They paused five days.

They changed tomorrow's meal.

They tried to resume a delivery after the cut-off.

They looked at meals before Delicut had assigned them.

They changed the delivery address for only one day.

They returned after their subscription had ended.

Every one of these actions changed what the app should show and what the customer should be allowed to do.

The problem was no longer:

“What should the home screen look like?”

It became:

“What should the home screen do in every possible state of a subscription?”

Initial findings

As we worked through the existing product, three problems kept appearing.

1. Too much product knowledge was required from the customer

Customers had to understand things like assignment dates, delivery cut-offs, skipped days and subscription rules before knowing what actions were available.

The interface needed to explain these rules through behaviour instead of expecting people to remember them.

---

2. Similar screens could behave very differently

A meal card for today might be view-only.

The same card tomorrow might allow the customer to change the meal, skip the delivery or update the address.

A card from yesterday might instead ask for a rating.

The visual difference could be small, but the product rules were very different.

---

3. The edge cases were actually normal behaviour

What looked like “edge cases” at first became a large part of the product.

Our home-screen mapping eventually covered **27 different situations**, from active and expired plans to skipped days, mixed weeks, cut-off states, meal changes and future assignments.

That changed how I thought about the project.

These were not exceptions we could design later.

They were the product.

My goals

I wanted the experience to do four things well.

Make the next step obvious

A customer should quickly understand what is happening and what they can do next.

Hide complexity where possible

The system could have complicated rules without making the interface feel complicated.

Keep behaviour consistent

The same product rule should lead to the same interface pattern wherever it appeared.

Design for change

Delicut was evolving quickly. The system needed to support new plans, new subscription rules and future features without redesigning everything from scratch.

Design process

01 — First, we simplified how people bought a plan

The subscription flow had several kinds of customers.

Some knew their health goals.

Some did not.

Some had custom macro targets.

Others wanted more control over their meals.

We explored different entry points for these needs instead of forcing everyone through the exact same path.

The goal was still the same:

help people understand what they should choose and why.

---

02 — We changed the web journey to follow a familiar shopping model

An early version of the subscription experience felt more like filling out a product form.

We later reorganised it around a model people already understood:

Select → Delivery details → Review → Payment

This borrowed from common e-commerce behaviour.

The benefit was not visual.

It reduced the amount of learning required.

People could focus on choosing their meal plan instead of figuring out how Delicut's checkout worked.

---

03 — We reduced early sign-up friction

A traditional product might ask:

Create an account

before allowing someone to continue.

But at that point the customer had not yet completed the main task they came for.

We instead treated contact information as part of the normal purchase journey.

The customer could move from:

landing page → plan selection → personal details → checkout

without an artificial sign-up interruption.

Their account could form naturally around information they already needed to provide.

Designing the mobile experience

Once a customer had subscribed, the design problem changed.

The app was no longer helping them choose.

It was helping them manage something that changed every day.

---

04 — I mapped the subscription as a system of states

Instead of immediately designing screens, we mapped what could be true when someone opened the app.

For example:

Plan
Not started / Active / Expiring / Expired

Day
Past / Today / Future

Delivery
Active / Skipped / Paused / Resumed

Meal
Not assigned / Assigned / Changed / Delivered

Cut-off
Editable / Locked

Action
View / Change / Skip / Resume / Rate / Renew

These states could combine.

That is where the real complexity appeared.

A future delivery could normally be edited.

But not if its cut-off had passed.

A skipped delivery could normally be resumed.

But not after the resume cut-off.

A past meal could normally be rated.

But not if that day had been skipped.

Mapping these combinations helped us move from individual screens to product rules.

One interface, different moments

I wanted the experience to make the customer's position in the subscription feel obvious.

So the interface changed based on three things:

Where am I in my plan?

Which day am I looking at?

What can I still do?

This created a simple pattern for several very different situations.

Before the plan begins

New customer
Plan not started
Upcoming

The customer has finished setup, but the subscription starts in a few days.

There is very little they need to do.

Instead of filling the screen with controls, the app shows the important dates, what happens next and when the first delivery will arrive.

The experience is intentionally calm.

Today

Active plan
Today
Locked

Once today's cut-off has passed, the customer can see their assigned meals and delivery information but cannot change them.

The key design decision here was not adding another feature.

It was making the lack of an action clear.

The product needed to say:

This is what is happening today.

without suggesting that the customer could still edit it.

Tomorrow

Active plan
Future day
Editable

Tomorrow can behave very differently.

Before the cut-off, the customer may be able to:

change a meal,
skip the delivery,
pause future deliveries,
or change the delivery address.

The interface uses the same meal information but opens the actions that are still possible.

This made the product rules visible through the interface itself.

Yesterday

Active plan
Past day
Rate

Past deliveries no longer need management controls.

They become feedback moments.

The customer can see what was delivered and rate the meal.

If the day had been skipped, those rating actions disappear because no meal was delivered.

One date change completely changes the purpose of the screen.

Skip and resume needed their own system

Skipping sounds like a simple feature.

It was not.

Customers could:

skip one day,
skip several days,
skip their entire remaining plan,
schedule a future skip,
resume some days but not others,
or try to resume after the cut-off.

The original model treated skip and resume as separate actions.

That made it difficult to understand the actual state of each day.

We moved toward a calendar-based model where the customer could understand delivery status day by day.

Instead of asking:

“Did I pause my plan?”

the interface could answer:

“Which days will I receive food?”

That is a much simpler mental model.

Designing around cut-offs

One of the most important product constraints was time.

Some actions were only possible before a delivery cut-off.

After that, meals were locked.

I did not want customers to discover these rules only after pressing a button.

So editable and locked states needed to look and behave differently before the action happened.

Examples included:

Future + before cut-off
Change meal / Skip / Change address

Future + after cut-off
View only

Skipped + before cut-off
Resume delivery

Skipped + after cut-off
Too late to resume

The constraint stayed complex.

The experience did not have to.

Meals were not always available yet

Another time-based state happened before Delicut assigned the weekly meals.

A customer could open a future date even though the menu for that week had not been generated yet.

Showing an empty meal card would look like an error.

Instead, the product explains what is happening:

Meals will be available after Wednesday at 6 PM.

This is a small example of a principle we used throughout the product:

when the system knows why something is unavailable, tell the customer.

Expiry was a journey, not one screen

A subscription does not suddenly become irrelevant on its final day.

We designed separate states around the end of the plan.

Expiring soon

When only a short time remains, the customer can see that their plan is approaching its end.

This creates space for renewal before their deliveries stop.

Expired

After the plan ends, future meal controls disappear.

The product shifts toward one clear next step:

Renew plan

This gave returning customers a way back into Delicut without making the app feel broken or empty.

Simplifying the wellness dashboard

The mobile app initially explored more detailed calorie tracking and calorie maths.

That created another problem.

Delicut wanted to support people who were beginning healthier routines, not make them feel like they needed to understand nutrition calculations.

We moved toward simpler, more useful signals such as:

steps

meals

streaks

weight

The idea was to focus the dashboard on actions people could understand and repeat.

Less calculation.

More progress.

Onboarding

The onboarding flow gathers only the information needed to make the experience useful.

The customer confirms:

activity level

height

current weight

target weight

They can then connect a fitness platform such as Apple Health, Google Fit, Fitbit, Garmin, Peloton, Oura or Huawei Health.

After setup, the app does not immediately throw them into an empty dashboard.

If their subscription starts later, it explains what happens next and shows the first important dates.

The onboarding therefore ends with context, not just completion.

Designing while the product was still changing

A major challenge was not visible in the final interface.

The product direction changed often.

The client was deeply involved in the design process and was learning Figma while we worked together. Ideas could appear during calls, existing screens could change quickly and AI tools were sometimes used live to question or suggest interface decisions.

That created a risk:

every new idea could become a new design direction.

We needed a better way to work.

---

05 — I learned to move conversations from opinion to rationale

At first, it was easy for reviews to become reactive.

Someone suggested a change.

We changed the screen.

Another idea appeared.

The screen changed again.

The turning point was documenting ideas before treating them as decisions.

For important changes, we started asking:

What problem are we solving?

What does the customer need here?

What product rule is involved?

Does this change affect another state?

Can we solve it with an existing pattern?

That created a shared reference point.

It also made it easier to disagree without making the conversation personal.

The discussion could return to the customer and the product logic.

Constraints

The rules were connected

Changing one delivery rule could affect several screens.

A new cut-off rule might affect:

home,
deliveries,
meal cards,
skip states,
resume states,
and notifications.

I had to think beyond the screen I was currently designing.

Web and mobile had different jobs

The web experience was mainly about **choosing and buying**.

The mobile app was mainly about **managing and continuing**.

Trying to make both platforms behave the same would have created unnecessary complexity.

The product was moving while we designed it

Flows, labels and business decisions continued to change during implementation.

The design system therefore needed enough consistency to absorb changes without starting over.

Development created new constraints

Not every design could move directly from Figma into the product.

During QA, issues ranged from visual spacing and typography to broken navigation, missing states, incorrect content and flow logic across both Android and iOS.

This made implementation review part of the design work, not an afterthought.

Design QA

I stayed involved after the main screens were designed.

The QA process covered:

Visual/UI
spacing, typography, alignment and component differences

Functional UX
incorrect behaviour, unavailable actions and navigation problems

Interaction
feedback, active states and gestures

Content
incorrect labels and missing guidance

Flow logic
missing states and incorrect transitions

The issue log also separated serious flow-breaking problems from smaller visual differences and design suggestions.

That helped us discuss implementation problems by impact rather than treating every mismatch as equally important.

Outcome

The biggest outcome was not a single redesigned screen.

It was a clearer product model.

We moved Delicut toward an experience where:

- subscription states were defined instead of handled one screen at a time
- past, present and future days had clear roles
- editable and locked deliveries behaved differently
- skip and resume worked around individual delivery days
- expiry and renewal became part of the subscription lifecycle
- web checkout followed a more familiar shopping pattern
- onboarding led naturally into the customer's actual plan state
- mobile metrics became simpler and more action-focused
- design QA connected the intended experience with what was actually being built

The home-screen work alone documented 27 situations the product needed to handle.

That gave the team a clearer base for both the current app and future product decisions.

What I learned

The most important lesson from Delicut was that complexity does not disappear because the interface looks simple.

Someone still has to understand it.

For this project, much of my work happened before drawing the final screen:

mapping states,
finding conflicting rules,
asking what should happen next,
documenting decisions,
and connecting one flow to another.

I also learned how important it is to stay steady when a project changes quickly.

A designer cannot stop every new idea.

But I can help the team understand which ideas solve a real problem, which create new problems and which should wait.

That changed how I see product design.

My job is not only to make the interface clear.

It is also to help make the product itself clearer.